Laporan UTS Graf Sains Data

Nama: Alamsyah Ramadhan Vaganza

NIM: 23031554192

Kelas: 2023B (no. 32)

Pada UTS ini saya mengimpor data interaksi RetailRocket (`events.csv`) ke Neo4j sebagai graf bipartit, dengan Visitor sebagai user dan Item sebagai produk. Relasinya ada tiga jenis, yaitu `VIEWED`, `ADDED_TO_CART`, dan `BOUGHT`. Dari graf ini saya menghitung kemiripan antar-item dengan cosine similarity, lalu membuat rekomendasi untuk satu user memakai KNN.

Data dibagi menjadi dua periode sesuai ketentuan yang diberikan untuk saya (no. 32):

Periode	Mulai	Hingga	Event	Visitor	Item
P1 (training)	01/06/2015	01/08/2015	1.323.616	684.281	171.044
P2 (testing)	10/08/2015	25/08/2015	286.038	164.220	79.628

Setiap jenis interaksi diberi bobot berbeda: view 1, addtocart 3, dan transaction 5. Pembelian jauh lebih jarang daripada view, jadi bobotnya dibuat lebih besar agar tidak tenggelam. Untuk semua proses acak, saya memakai NIM sebagai seed.

1. Tugas Pendahuluan

Langkah awalnya adalah menyiapkan environment Python dan menjalankan Neo4j secara lokal di `neo4j://127.0.0.1:7687`. Koneksi dari Python berhasil, dan tutorial `add_friend` mencetak Guinevere, Lancelot, dan Merlin seperti yang diharapkan. Saya memakai driver `neo4j>=5.8` karena fungsi `execute_query` dan `RoutingControl` di tutorial belum tersedia di driver versi 4.2.1 yang tertulis pada `requirements.txt` soal.

Jawaban pertanyaan checkpoint:

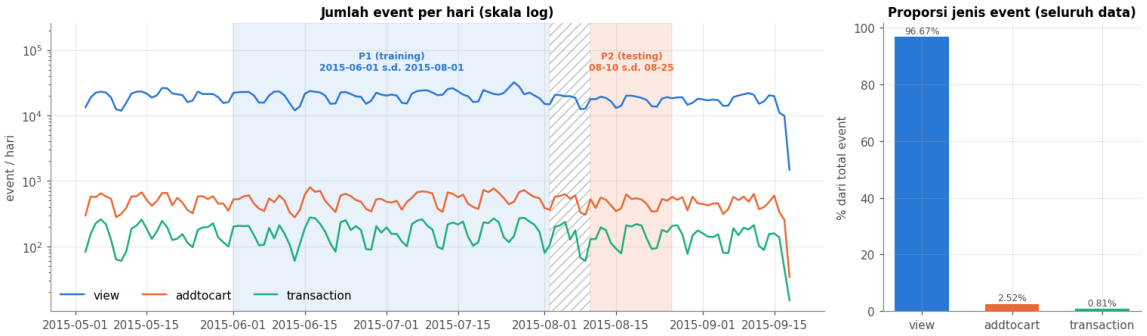
1. Database yang dipakai bernama `neo4j`. Nama ini ditentukan lewat parameter `database_="neo4j"` di setiap pemanggilan `execute_query`.
2. Untuk mengakses database lain dalam instance yang sama, cukup ganti nama database di parameter `database_` atau di `driver.session(database="nama")`. Di Cypher bisa memakai `USE nama`, sedangkan di Neo4j Browser memakai `:use nama`. Pengelolaan database dilakukan dari database `system` dengan perintah `SHOW`, `CREATE`, `STOP`, `START`, dan `DROP DATABASE`. Perintah `CREATE` dan `DROP` hanya tersedia di Enterprise Edition.
3. Graf pertemanan yang terbentuk adalah graf berarah dan tidak berbobot, dengan satu jenis relasi `KNOWS`. Bentuknya seperti bintang: Arthur berada di tengah dengan tiga relasi keluar ke Guinevere, Lancelot, dan Merlin.

Data kemudian diimpor ke Neo4j per batch 10.000 baris. Setiap batch membuat node dengan `MERGE` dan relasi dengan `CREATE`, ditambah constraint unik pada `visitorId` dan `itemId` agar tidak ada node ganda. Seluruh 1.609.654 baris selesai diimpor dalam 161 batch, kurang lebih 4 menit.

2.1 Wrangling dan EDA

No. 1 - Periode

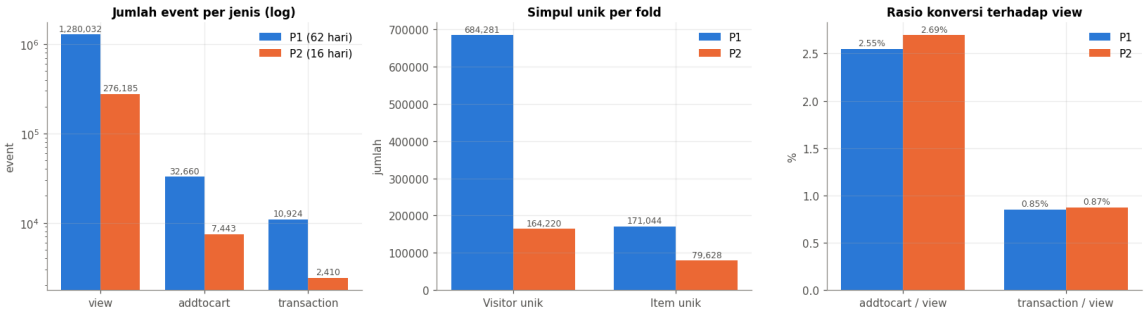
Periode P1 dan P2 saya ambil dari ketentuan dosen. Keduanya tidak beririsan, dan ada jeda 2 sampai 9 Agustus yang sengaja tidak dipakai supaya data uji tidak ikut terbaca saat model belajar.



Seluruh data berisi 2.756.101 event dari 3 Mei sampai 18 September 2015. Hampir semuanya berupa `view` (96,67%), sementara `addtocart` hanya 2,52% dan `transaction` 0,81%. Rata-rata event per hari di P1 (21.352) dan P2 (17.882) tidak jauh berbeda. Jadi P1 jauh lebih besar terutama karena periodenya lebih panjang, 62 hari dibanding 16 hari.

No. 2 - Filter dan simpan CSV

Data difilter berdasarkan timestamp, dengan tanggal akhir ikut dihitung. Hasilnya disimpan ke `data/events_filtered.csv` sebanyak 1.609.654 baris (sekitar 58% dari seluruh data, 61,5 MB), dan setiap baris diberi kolom `fold` berisi P1 atau P2.



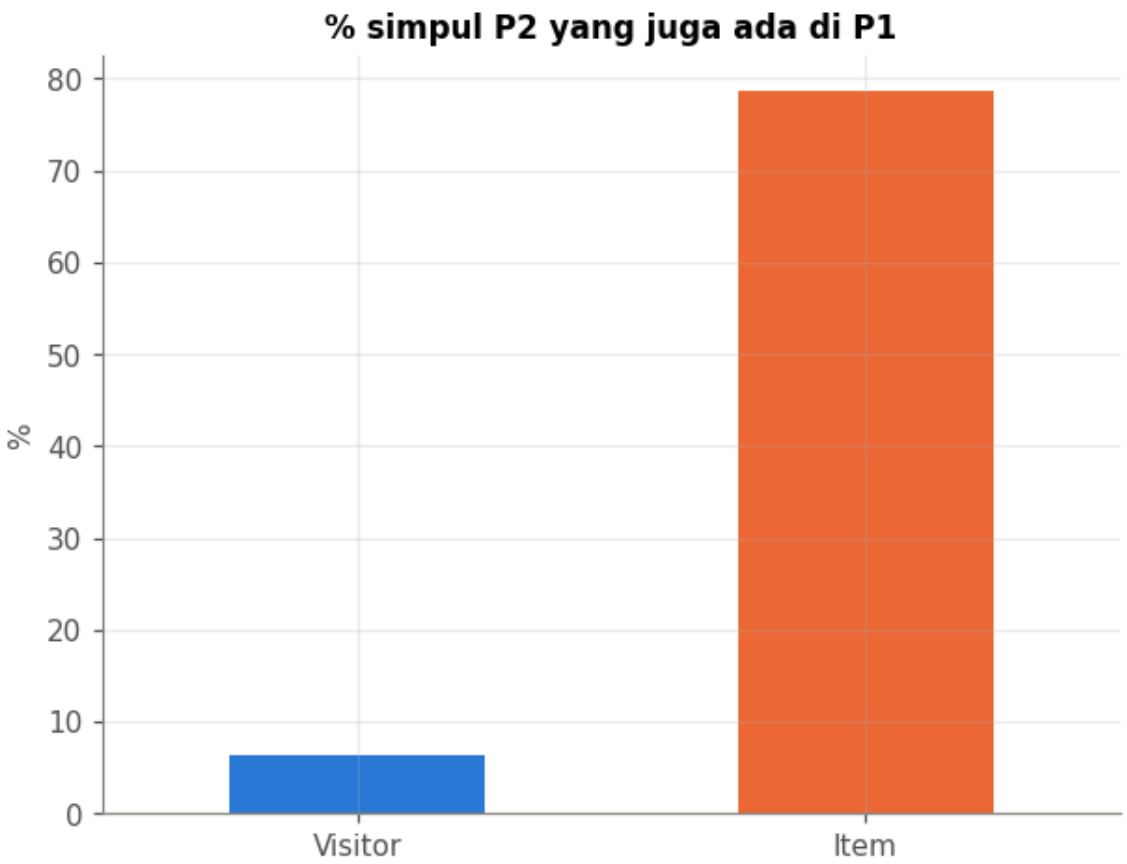
Perilaku pengunjung di kedua periode ternyata mirip. Rasio addtocart terhadap view adalah 2,55% di P1 dan 2,69% di P2, sedangkan rasio transaksinya 0,85% dan 0,87%. Ini penting, karena berarti data latih dan data uji punya karakter yang sama sehingga evaluasinya adil.

No. 3 - Komposisi subgraf

Subgraf yang terbentuk berisi 1.026.270 simpul dan 1.609.654 relasi, sama persis dengan jumlah baris CSV, jadi tidak ada data yang hilang saat impor. Graf ini bipartit dan berarah: semua relasi mengarah dari Visitor ke Item, dan tidak ada relasi antar-Visitor maupun antar-Item.

Relasi	P1	P2
`VIEWED`	1.280.032	276.185
`ADDED_TO_CART`	32.660	7.443
`BOUGHT`	10.924	2.410
Visitor / Item aktif	684.281 / 171.044	164.220 / 79.628

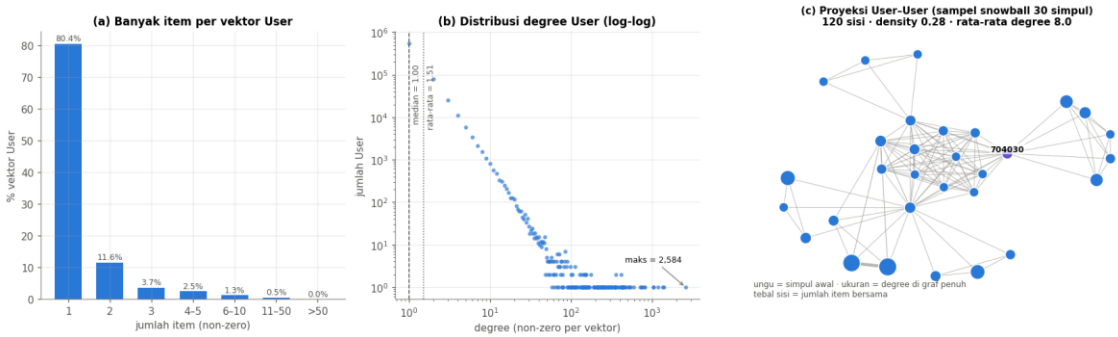
Secara total ada 838.210 Visitor dan 188.060 Item yang unik. Yang menarik terlihat dari grafik di bawah: hanya 10.291 visitor (6,3% dari visitor P2) yang juga muncul di P1, sedangkan 78,6% item P2 sudah ada di P1. Artinya sebagian besar pengunjung di masa uji adalah orang baru, sementara katalog produknya relatif tetap. Inilah masalah *cold-start user* yang nantinya memengaruhi hasil rekomendasi.



Query Cypher yang dipakai cukup sederhana. `MATCH (n:Visitor) RETURN count(n)` dan versi yang sama untuk `Item` menghitung jumlah simpul. `MATCH ()-[r]->() RETURN type(r), r.fold, count(*)` menghitung relasi per jenis dan per periode. Terakhir, `count(DISTINCT v)` dan `count(DISTINCT i)` yang dikelompokkan per `r.fold` menghitung berapa visitor dan item yang aktif di tiap periode. Semua query dijalankan dari Python lewat `driver.execute_query`.

No. 4 - VSM simpul User

Setiap user direpresentasikan sebagai vektor di ruang item. Isi setiap sel adalah jumlah bobot interaksi user terhadap item tersebut selama P1. Hasilnya matriks berukuran 684.281 × 171.044.

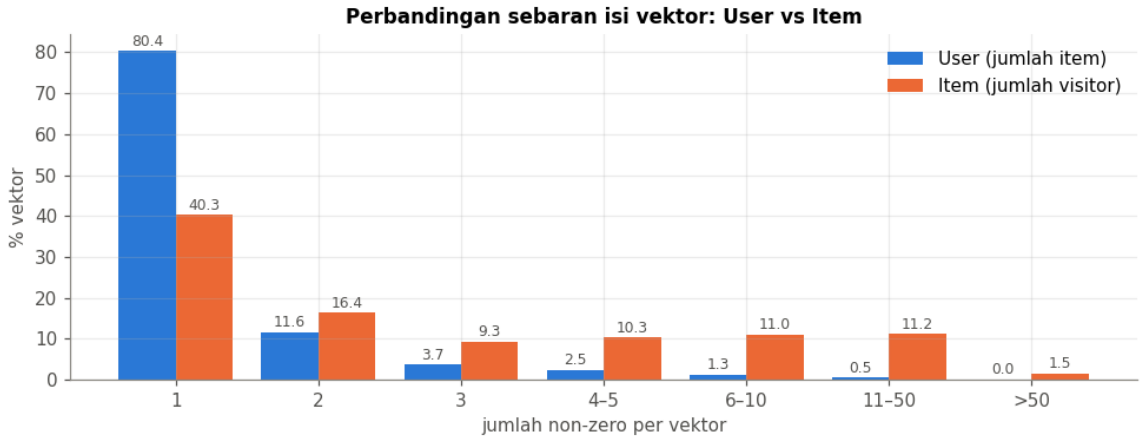


Matriks ini sangat kosong. Densitasnya hanya $8{,}8 \times 10^{-6}$, jadi kalau disimpan utuh butuh sekitar 468 GB, sedangkan format sparse CSR cukup 15,1 MB. Sebanyak 80,4% user hanya pernah menyentuh satu item, dengan rata-rata 1,51 item per user. Distribusinya membentuk garis lurus di grafik log-log, ciri khas pola *long-tail*: banyak sekali user dengan sedikit interaksi dan segelintir user yang sangat aktif. Ada satu user dengan 2.584 item, yang kemungkinan besar adalah bot.

Pada graf proyeksi User-User (grafik kanan), dua user terhubung kalau pernah melihat item yang sama. Karena kebanyakan user hanya punya satu item, hubungan ini sering terjadi hanya karena kebetulan melihat satu produk yang sama.

No. 5 VSM simpul Item

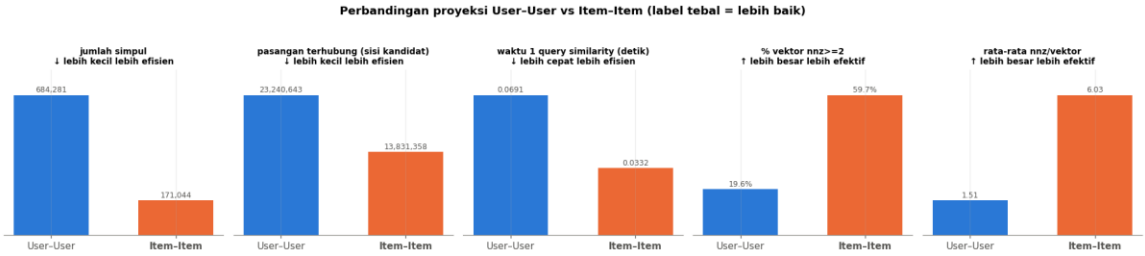
Vektor item adalah kebalikannya: setiap item direpresentasikan di ruang user, dengan ukuran 171.044 × 684.281. Jumlah nilai yang terisi sama, tetapi karena jumlah itemnya empat kali lebih sedikit, setiap vektor item jauh lebih padat.



	VSM User	VSM Item
Ukuran matriks	684.281 × 171.044	171.044 × 684.281
Nilai terisi (non-zero)	1.031.265	1.031.265
Rata-rata / median isi per vektor	1,51 / 1	6,03 / 2
Isi terbanyak dalam satu vektor	2.584	1.616 (item 5411)
Vektor yang hanya berisi 1 nilai	80,4%	40,3%

Hanya 40,3% item yang dilihat oleh satu visitor saja, jauh di bawah angka 80,4% pada user. Item terpopuler adalah 5411 dengan 1.616 visitor, dan item ini yang nanti dipakai sebagai acuan di no. 7. Graf proyeksi Item-Item dengan networkx ada di notebook dan terlihat lebih rapat dibanding proyeksi User-User.

No. 6 - Proyeksi yang dipilih: Item-Item



Dari perbandingan di atas, proyeksi Item-Item lebih baik di semua indikator. Jumlah simpulnya empat kali lebih sedikit (171 ribu dibanding 684 ribu), pasangan yang harus dihitung 40% lebih sedikit (13,8 juta dibanding 23,2 juta), dan waktu hitung similarity sekitar dua kali lebih cepat (0,033 detik dibanding 0,069 detik). Vektornya juga lebih padat: 59,7% item punya minimal dua visitor, sedangkan user yang punya minimal dua item hanya 19,6%.

Jadi proyeksi Item-Item lebih efisien karena biaya hitungnya kecil, dan lebih efektif karena vektornya cukup terisi untuk menghasilkan similarity yang stabil. Selain itu, katalog item lebih stabil antar-periode (78,6% item P2 sudah ada di P1). Karena itu proyeksi Item-Item yang saya pakai untuk no. 7 dan 8.

No. 7 - Similarity pada proyeksi Item-Item

Ukuran kemiripan yang saya pakai adalah cosine similarity berbobot. Item acuannya 5411, karena item ini punya visitor terbanyak di P1 (1.616 visitor). Item yang paling mirip dengannya adalah 33030 dengan nilai cosine 0,1655.

```
\cos(a,b)=\frac{\sum_v w_{va}\,w_{vb}}{\lVert a\rVert\,\lVert b\rVert}
```

(a) Query Cypher untuk 5 record teratas. Sebelum query dijalankan, panjang (norma) vektor setiap item disimpan dulu sebagai properti `norm_p1`.

```
MATCH (ref:Item {itemId: 5411})<-[e1]-(v:Visitor) WHERE e1.fold = 'P1'
WITH ref, v, sum(CASE type(e1) WHEN 'VIEWED' THEN 1 WHEN 'ADDED_TO_CART' THEN
3 WHEN 'BOUGHT' THEN 5 END) AS wr
MATCH (v)-[e2]->(o:Item) WHERE e2.fold = 'P1' AND o <> ref
WITH ref, o, v, wr, sum(CASE type(e2) WHEN 'VIEWED' THEN 1 WHEN
'ADDED_TO_CART' THEN 3 WHEN 'BOUGHT' THEN 5 END) AS wo
WITH ref, o, sum(wr * wo) AS dot
RETURN o.itemId AS item, round(dot / (ref.norm_p1 * o.norm_p1), 6) AS cosine
ORDER BY cosine DESC LIMIT 5
```

Query ini bekerja dalam tiga langkah. Pertama, ambil semua visitor P1 yang pernah berinteraksi dengan item 5411 beserta bobotnya (`wr`). Kedua, dari visitor yang sama, ambil item lain yang mereka lihat beserta bobotnya (`wo`); langkah ini pada dasarnya adalah proyeksi Item-Item. Ketiga, jumlahkan `wr × wo`, lalu bagi dengan perkalian kedua norma.

Peringkat	Item	Cosine (Python)	Cosine (Cypher)	Visitor yang sama
1	33030	0,165460	0,165460	66
2	256242	0,127863	0,127863	1
3	33956	0,108185	0,108185	19
4	37807	0,102079	0,102079	15
5	459936	0,090442	0,090442	31

Hasil dari Cypher sama persis dengan hasil Python sampai 6 angka di belakang koma, jadi perhitungannya bisa dipercaya. Nilai cosine-nya memang kecil (di bawah 0,17) karena item 5411 punya sangat banyak visitor sehingga normanya besar.

Ada satu hal yang perlu dicatat. Item 256242 berada di peringkat 2 padahal hanya punya satu visitor yang sama dengan item 5411. Ini terjadi karena item 256242 hampir tidak pernah dilihat orang lain, sehingga normanya kecil dan nilai cosine-nya ikut terangkat. Ini kelemahan cosine pada item yang jarang dilihat. Dalam praktik, biasanya ditambahkan syarat minimal jumlah visitor yang sama.

(b) Menulis similarity ke graf. Kelima nilai tersebut disimpan kembali ke Neo4j sebagai relasi `(:Item)-[:SIMILAR_TO {score, method, fold}]->(:Item)` memakai `MERGE`. Sebelumnya graf tidak punya relasi `SIMILAR_TO` sama sekali, dan setelah query dijalankan ada 5 relasi baru dengan skor yang sama seperti tabel di atas.

No. 8 - Sebelum dan sesudah perhitungan similarity

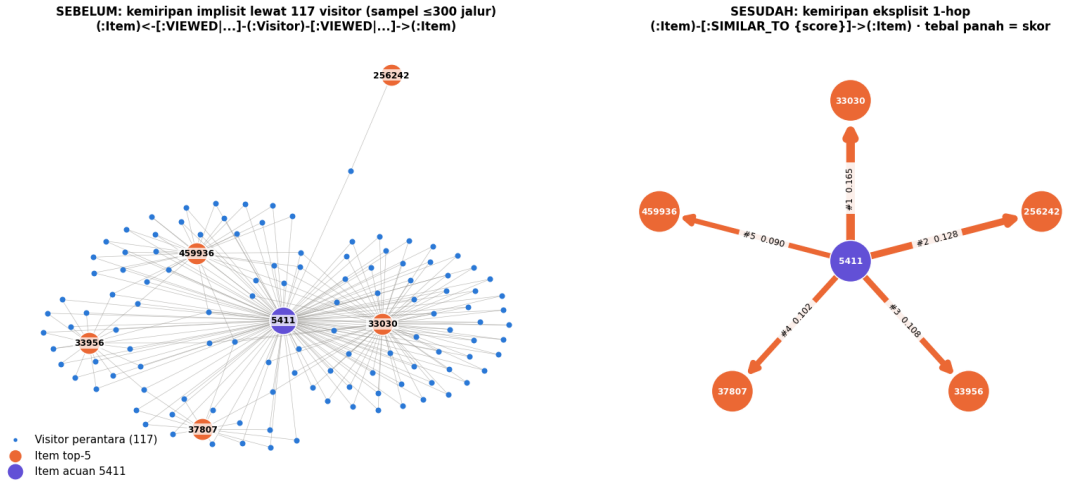
Sebelum similarity dihitung, kemiripan antar-item hanya bisa dilihat secara tidak langsung, yaitu lewat jalur dua langkah `Item ← Visitor → Item`. Setelah dihitung, kemiripan itu tersimpan sebagai relasi langsung `Item → Item` lengkap dengan nilainya.

Query untuk kondisi sebelum:

```
MATCH p = (a:Item {itemId: 5411})<-[e1]-(v:Visitor)-[e2]->(b:Item)
WHERE b.itemId IN [33030, 256242, 33956, 37807, 459936]
  AND e1.fold = 'P1' AND e2.fold = 'P1'
RETURN p LIMIT 300
```

Query untuk kondisi sesudah:

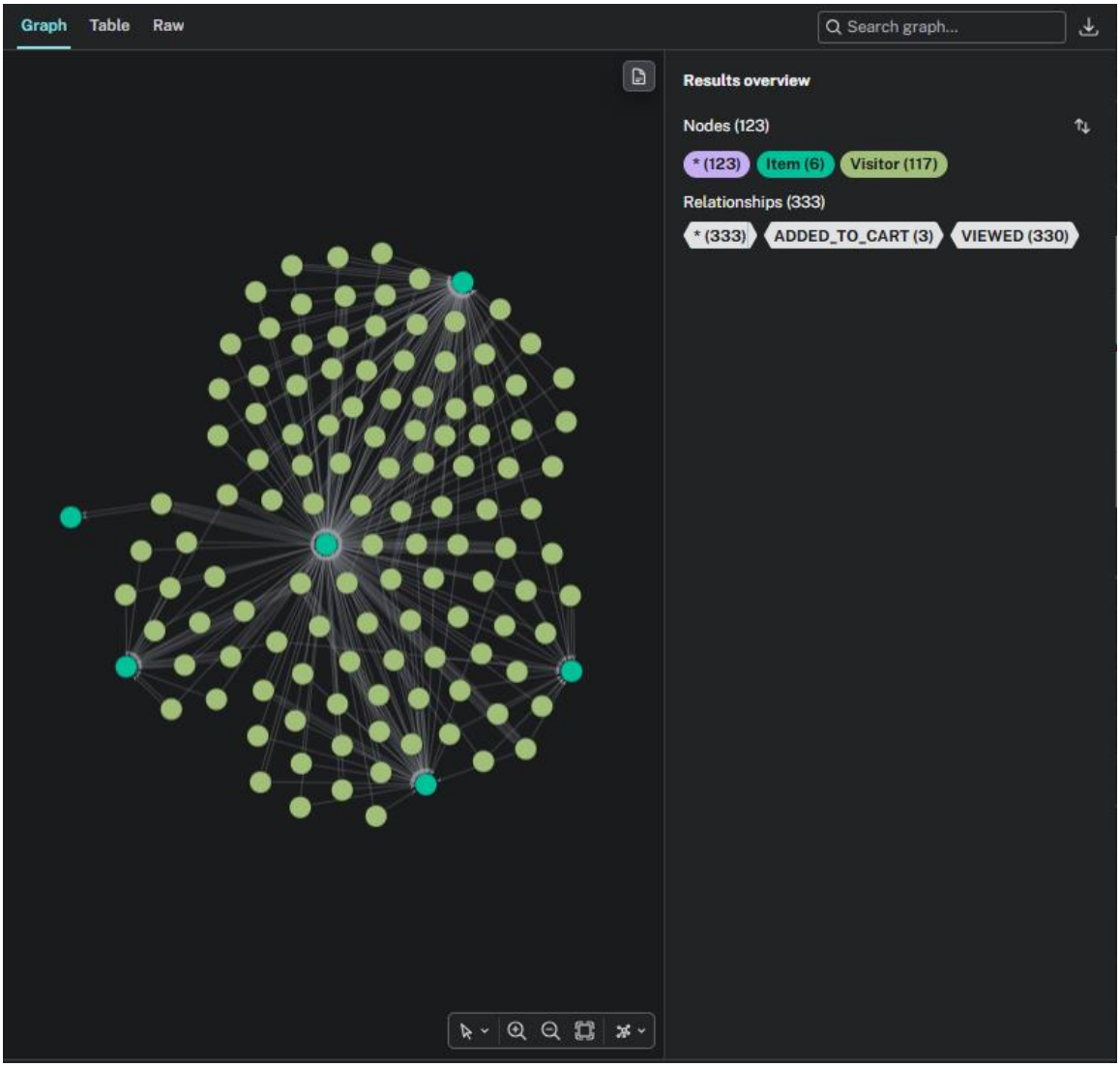
```
MATCH p = (a:Item {itemId: 5411})-[s:SIMILAR_TO]->(b:Item)
RETURN p
```

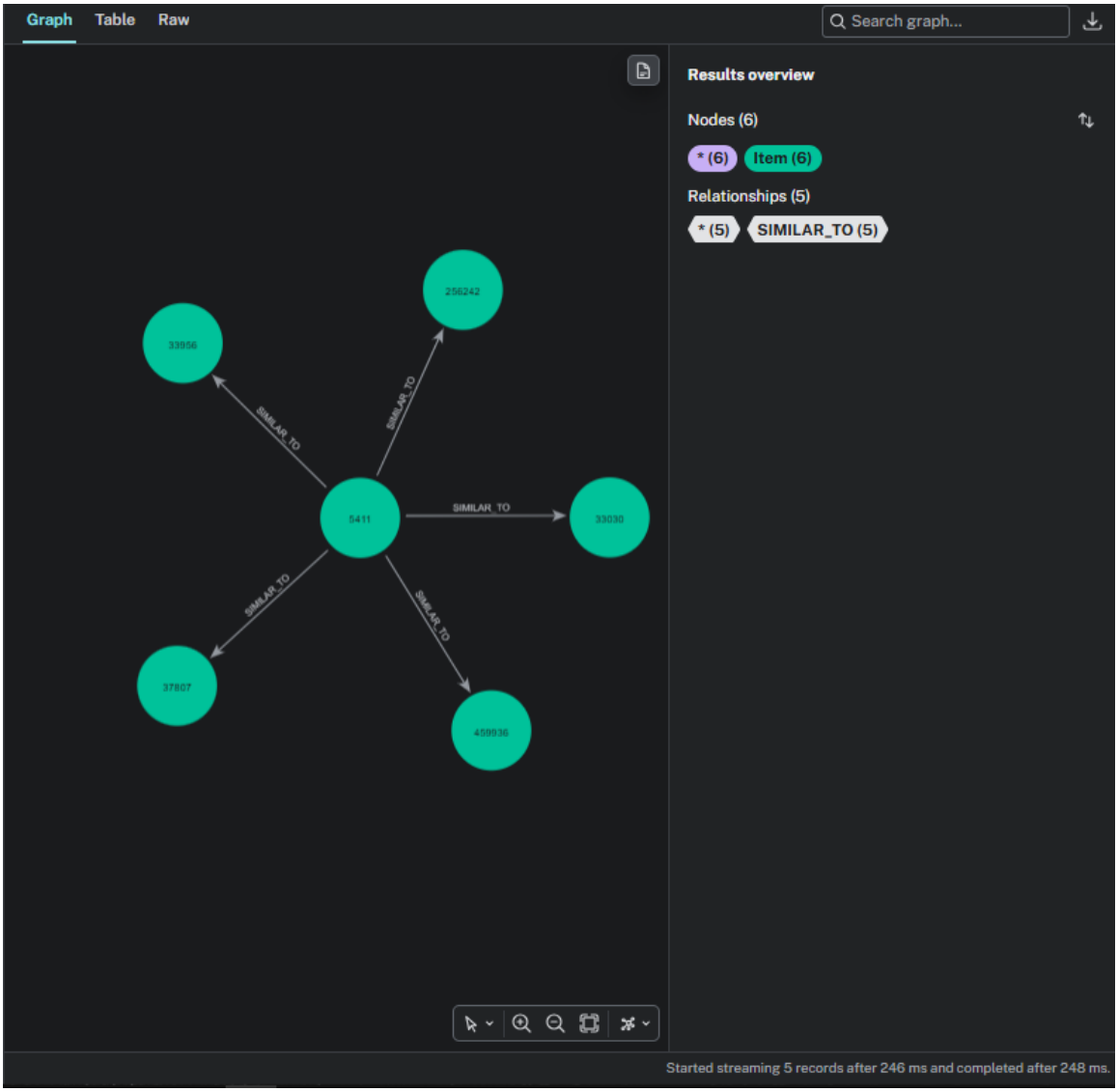


	Sebelum	Sesudah
Pola jalur	Item ← Visitor → Item (2 langkah)	Item → Item (1 langkah)
Yang ditelusuri	1.616 visitor dan semua item mereka	5 relasi
Waktu query (median 3 kali)	81,5 ms	22,9 ms

Waktu query turun dari 81,5 ms menjadi 22,9 ms, sekitar 3,6 kali lebih cepat, karena tidak perlu lagi menelusuri ribuan visitor. Graf yang awalnya murni bipartit kini punya hubungan langsung antar-item yang bisa dipakai, misalnya untuk fitur "produk serupa".

Berikut tampilan yang sama di Neo4j Browser. Kondisi sebelum menampilkan 123 simpul dan 333 relasi, sedangkan kondisi sesudah hanya 6 simpul dengan 5 relasi `SIMILAR_TO`.


Graph
Table
Raw
Search graph...
Results overview
Nodes (123)
* (123)
Item (6)
Visitor (117)
Relationships (333)
* (333)
ADDED_TO_CART (3)
VIEWED (330)



2.2 Analytics

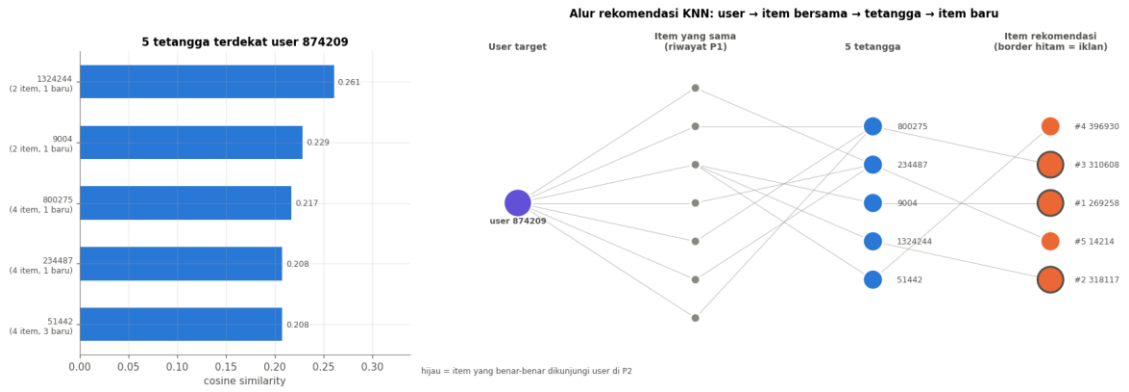
No. 1 - User acak dan rekomendasi KNN

User yang terpilih adalah 874209, dan tiga item yang diiklankan untuknya adalah 269258, 318117, dan 310608.

Cara memilih user acaknya begini. Pertama saya kumpulkan kandidat, yaitu user yang punya minimal 3 item di P1 dan juga aktif di P2, supaya hasilnya bisa dievaluasi. Ada 2.626 kandidat. Kandidat ini diurutkan, lalu satu dipilih memakai `np.random.default_rng(SEED)` dengan SEED berupa NIM saya (23031554192). Karena seed-nya tetap, user yang terpilih akan selalu sama setiap kali kode dijalankan.

Rekomendasinya memakai user-based KNN dengan cosine similarity dan k = 5:

1. Calon tetangga diambil dari 134.420 user P1 yang punya minimal 2 item. User dengan satu item tidak dipakai karena tidak bisa menyumbang item baru.
2. Dari calon tersebut, dipilih 5 user paling mirip yang punya setidaknya satu item yang belum pernah dilihat user target.
3. Setiap item dari tetangga diberi skor, yaitu jumlah cosine tetangga dikali bobot interaksinya. Item yang sudah pernah dilihat user target tidak dihitung.
4. Item diurutkan berdasarkan skor, dan tiga teratas menjadi item yang diiklankan.



Peringkat	Item	Skor	Diiklankan
1	269258	0,4572	Ya
2	318117	0,2606	Ya
3	310608	0,2172	Ya
4	396930	0,2077	Tidak
5	14214	0,2077	Tidak

User 874209 punya 50 item di P1. Kelima tetangganya punya nilai cosine antara 0,208 dan 0,261. Item 269258 mendapat skor tertinggi karena tetangga 9004 (cosine 0,2286) berinteraksi dengannya dengan bobot 2, sehingga skornya 0,2286 × 2 = 0,4572.

No. 2 - Hit@5 dan NDCG@5

Hasilnya Hit@5 = 0 dan NDCG@5 = 0. Di P2, user 874209 hanya mengunjungi satu item, yaitu 95318, dan item ini tidak termasuk dalam lima rekomendasi.

Hit@5 bernilai 1 kalau minimal satu item rekomendasi benar-benar dikunjungi di P2. NDCG@5 juga memperhitungkan posisinya, jadi item yang tepat di peringkat atas dihargai lebih tinggi. Karena tidak ada satu pun yang tepat, kedua nilai sama dengan 0.

Hasil ini masih masuk akal kalau melihat kondisi datanya. Rekomendasi harus menebak satu item yang tepat dari sekitar 171 ribu item hanya dengan lima pilihan. Tetangga yang ditemukan juga tidak terlalu mirip (cosine paling tinggi 0,26), karena 80% user hanya punya satu item.

Ditambah lagi, 96,7% interaksi hanya berupa view, yang sinyalnya lebih lemah daripada pembelian.

Kesimpulan

Proyeksi Item-Item terbukti lebih efisien dan efektif dibanding User-User. Similarity yang dihitung di atasnya konsisten antara Python dan Cypher, dan setelah disimpan sebagai relasi `SIMILAR_TO`, item yang mirip bisa langsung dibaca dengan query satu langkah.

Rekomendasi KNN untuk user 874209 menghasilkan iklan 269258, 318117, dan 310608, tetapi belum ada yang tepat (Hit@5 dan NDCG@5 sama dengan 0). Penyebab utamanya adalah data yang sangat jarang terisi dan banyaknya pengunjung baru di periode uji, karena hanya 6,3% visitor P2 yang pernah muncul di P1. Untuk pengembangan berikutnya, rekomendasi bisa dicoba dengan item-based KNN memakai relasi `SIMILAR_TO`, atau digabung dengan popularitas item.